

Complete Code Explanation - NestJS TP02 Project

Table of Contents

- 1. [Project Overview](#)
- 2. [File Structure](#)
- 3. [Core Application Files](#)
 - [main.ts](#)
 - [app.module.ts](#)
 - [app.controller.ts](#)
 - [app.service.ts](#)
 - [app.controller.spec.ts](#)
- 4. [Database Layer](#)
 - [receipts.entities.ts](#)
- 5. [Receipts Feature Module](#)
 - [receipts.module.ts](#)
 - [receipts.controller.ts](#)
 - [receipts.service.ts](#)
- 6. [Data Transfer Objects \(DTOs\)](#)
 - [create-receipt.dto.ts](#)
 - [update-receipt.dto.ts](#)
- 7. [Common Utilities](#)
 - [api-key.guard.ts](#)
 - [logging.interceptor.ts](#)
- 8. [Code Relationships Diagram](#)
- 9. [Keyword Glossary](#)

Project Overview

This is a **NestJS** application that implements a **Receipt Management System** with a **PostgreSQL** database. The application follows the **MVC (Model-View-Controller)** architecture pattern and uses **TypeORM** as the Object-Relational Mapping (ORM) tool.

Key Technologies Used:

- **NestJS** - A progressive Node.js framework for building server-side applications
- **TypeORM** - Database ORM for TypeScript and JavaScript
- **PostgreSQL** - Relational database
- **class-validator** - Validation decorators for DTOs

File Structure

```
src/  
├── main.ts                                # Application entry point
```

```
├── app.module.ts           # Root module
├── app.controller.ts       # Root controller
├── app.service.ts          # Root service
├── app.controller.spec.ts  # Unit tests for app controller
├── common/
│   ├── guards/
│   │   └── api-key.guard.ts # API key authentication guard
│   ├── interceptors/
│   │   └── logging.interceptor.ts # HTTP request logging
├── database/
│   └── entities/
│       └── receipts.entities.ts # Receipt database entity
├── receipts/
│   ├── receipts.module.ts    # Receipts feature module
│   ├── receipts.controller.ts # Receipts API endpoints
│   ├── receipts.service.ts   # Receipts business logic
│   └── dto/
│       ├── create-receipt.dto.ts # DTO for creating receipts
│       └── update-receipt.dto.ts  # DTO for updating receipts
```

Core Application Files

1. `main.ts` - Application Entry Point

Location: `src/main.ts`

This file is the **starting point** of the entire NestJS application. It bootstraps and configures the application.

```
import { NestFactory } from '@nestjs/core';
```

Line 1 Explanation:

- `import` - JavaScript/TypeScript keyword to bring in external code modules
- `{ NestFactory }` - Destructuring import that extracts the `NestFactory` class from the NestJS core package
- `NestFactory` - A factory class provided by NestJS that creates an instance of the application
- `from '@nestjs/core'` - The source package (NestJS core library)
- **Why we need it:** Without `NestFactory`, we cannot create or start the NestJS application. This is the fundamental building block
- **If deleted:** The application will not compile because we cannot create the app instance

```
import { AppModule } from './app.module';
```

Line 2 Explanation:

- `AppModule` - The root module of our application that contains all configurations

- `'./app.module'` - Relative path to the app.module.ts file (`.ts` extension is omitted)
- **Why we need it:** The root module tells NestJS what controllers, services, and other modules to load
- **If deleted:** The application has no root module and won't know what to load

```
import { ValidationPipe } from '@nestjs/common';
```

Line 3 Explanation:

- `ValidationPipe` - A built-in NestJS pipe that validates incoming request data
- `@nestjs/common` - Common NestJS utilities package
- **Why we need it:** Automatically validates DTOs (Data Transfer Objects) against their decorators
- **If deleted:** Request validation won't work globally, and invalid data could enter the system

```
import { LoggingInterceptor } from  
'./common/interceptors/logging.interceptor';
```

Line 4 Explanation:

- `LoggingInterceptor` - Custom interceptor class that logs HTTP requests
- **Why we need it:** Provides visibility into all incoming requests and response times
- **If deleted:** No automatic logging of HTTP requests will occur

```
async function bootstrap() {
```

Line 8 Explanation:

- `async` - Keyword that marks this function as asynchronous (can use `await`)
- `function` - Keyword to declare a function
- `bootstrap` - Function name (conventional name for NestJS startup function)
- `()` - Empty parameters (no arguments needed)
- `{` - Opening brace for function body
- **Why we need it:** Wraps all startup logic in an async function to handle promises
- **If deleted:** Cannot use `await` for async operations like creating the app

```
const app = await NestFactory.create(AppModule);
```

Line 9 Explanation:

- `const` - Declares a constant variable (cannot be reassigned)
- `app` - Variable name holding the application instance
- `await` - Waits for the promise to resolve before continuing
- `NestFactory.create()` - Static method that creates a new NestJS application
- `AppModule` - The root module passed to configure the app

- **Why we need it:** Creates the actual application instance with all its configurations
- **If deleted:** No application instance exists, nothing can run

```
app.enableCors();
```

Line 11 Explanation:

- `app` - The application instance
- `.enableCors()` - Method that enables Cross-Origin Resource Sharing
- `CORS` - Security feature that allows/restricts cross-domain requests
- **Why we need it:** Allows frontend applications from different domains to access this API
- **If deleted:** Browsers will block requests from different origins (e.g., frontend on localhost:4200 cannot call backend on localhost:3000)

```
app.useGlobalInterceptors(new LoggingInterceptor());
```

Line 12 Explanation:

- `app.useGlobalInterceptors()` - Registers interceptors that run on every request
- `new` - Keyword to create a new instance of a class
- `LoggingInterceptor()` - Creates an instance of our logging interceptor
- **Why we need it:** Applies logging to ALL routes without decorating each controller
- **If deleted:** No automatic request logging occurs

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
  }),  
);
```

Lines 13-19 Explanation:

- `app.useGlobalPipes()` - Registers pipes that process all incoming data
- `new ValidationPipe({...})` - Creates a validation pipe with configuration
- `whitelist: true` - Automatically strips properties that don't have decorators in the DTO
- `forbidNonWhitelisted: true` - Throws an error if extra properties are sent
- `transform: true` - Automatically transforms payloads to DTO class instances
- **Why we need it:** Ensures all incoming data is validated and cleaned
- **If deleted:** DTOs won't be validated, invalid data can enter the system, security vulnerability

```
await app.listen(process.env.PORT ?? 3000);
```

Line 21 Explanation:

- `await` - Waits for the server to start listening
- `app.listen()` - Starts the HTTP server
- `process.env.PORT` - Environment variable for the port number
- `??` - Nullish coalescing operator (uses 3000 if PORT is null/undefined)
- `3000` - Default port number
- **Why we need it:** Starts the server and makes it accessible
- **If deleted:** Server never starts, application is useless

```
bootstrap();
```

Line 23 Explanation:

- Calls the `bootstrap` function to start the application
- **Why we need it:** Without calling the function, the app never starts
- **If deleted:** Application code exists but never executes

2. `app.module.ts` - Root Module

Location: `src/app.module.ts`

The **root module** is the central configuration hub of a NestJS application. It imports other modules and configures core features.

```
import { Module } from '@nestjs/common';
```

Line 1 Explanation:

- `Module` - A decorator function that marks a class as a NestJS module
- **Why we need it:** Required to define any module in NestJS
- **If deleted:** Cannot create the root module, application won't work

```
import { TypeOrmModule } from '@nestjs/typeorm';
```

Line 2 Explanation:

- `TypeOrmModule` - NestJS integration module for TypeORM
- **Why we need it:** Enables database connections and entity management
- **If deleted:** Cannot connect to the PostgreSQL database

```
import { AppController } from './app.controller';
```

Line 3 Explanation:

- **AppController** - The root controller handling base routes
- **Why we need it:** To register the controller with the module
- **If deleted:** Root endpoint `/` won't work

```
import { AppService } from './app.service';
```

Line 4 Explanation:

- **AppService** - The root service containing business logic
- **Why we need it:** Provides functionality to the AppController
- **If deleted:** AppController cannot call `getHello()` method

```
import { ReceiptsModule } from './receipts/receipts.module';
```

Line 5 Explanation:

- **ReceiptsModule** - Feature module for receipt management
- **Why we need it:** Brings in all receipt-related functionality
- **If deleted:** All receipt endpoints and features disappear

```
import { Receipt } from './database/entities/receipts.entities';
```

Line 6 Explanation:

- **Receipt** - The database entity class representing the receipts table
- **Why we need it:** TypeORM needs to know which entities exist for database synchronization
- **If deleted:** Database won't create the receipts table

```
import { ConfigModule } from '@nestjs/config';
```

Line 7 Explanation:

- **ConfigModule** - NestJS module for managing configuration and environment variables
- **Why we need it:** Loads and manages `.env` file variables
- **If deleted:** Environment variables like `DB_HOST` won't be automatically loaded

```
@Module({
```

Line 9 Explanation:

- **@Module** - Decorator that marks the class as a module
- **{** - Configuration object begins
- **Why we need it:** Tells NestJS this class is a module and how to configure it
- **If deleted:** Class is just a regular class, not recognized as a module

```
imports: [
```

Line 11 Explanation:

- **imports** - Array of other modules this module depends on
- **Why we need it:** Brings in functionality from other modules
- **If deleted:** No external modules are available

```
ConfigModule.forRoot({ isGlobal: true } ),
```

Line 12 Explanation:

- **ConfigModule.forRoot()** - Initializes the configuration module
- **isGlobal: true** - Makes config available throughout the entire app without re-importing
- **Why we need it:** Loads **.env** variables and makes them accessible via **process.env**
- **If deleted:** Environment variables from **.env** file won't be loaded

```
TypeOrmModule.forRoot({  
  type: 'postgres',  
  host: process.env.DB_HOST || 'localhost',  
  port: parseInt(process.env.DB_PORT || '5432'),  
  username: process.env.DB_USERNAME || 'postgres',  
  password: process.env.DB_PASSWORD || 'nakry123',  
  database: process.env.DB_NAME || 'tp02_db',  
  entities: [Receipt],  
  synchronize: true,  
  logging: false,  
}),
```

Lines 13-23 Explanation:

- **TypeOrmModule.forRoot({...})** - Configures the database connection
- **type: 'postgres'** - Database type (PostgreSQL)
- **host** - Database server address
- **port** - Database port (5432 is PostgreSQL default)
- **username** - Database user
- **password** - Database password
- **database** - Database name
- **entities: [Receipt]** - Array of entity classes to manage
- **synchronize: true** - Auto-creates/updates tables based on entities (disable in production!)

- `logging: false` - Disables SQL query logging
- `process.env.X || 'default'` - Uses environment variable or falls back to default
- `parseInt()` - Converts string to integer (environment variables are always strings)
- **Why we need it:** Establishes database connection with proper credentials
- **If deleted:** No database connection, application cannot store data

```
ReceiptsModule,
```

Line 24 Explanation:

- Imports the ReceiptsModule into the root module
- **Why we need it:** Makes all receipt features available in the application
- **If deleted:** Receipt endpoints won't exist

```
],  
controllers: [AppController],
```

Lines 25-26 Explanation:

- `controllers` - Array of controllers this module provides
- `[AppController]` - The root controller
- **Why we need it:** Registers controllers so their routes are accessible
- **If deleted:** No routes are registered for this module

```
providers: [AppService],
```

Line 27 Explanation:

- `providers` - Array of services/providers this module creates
- `[AppService]` - The root service
- **Why we need it:** Makes AppService injectable into controllers
- **If deleted:** Dependency injection for AppService fails

```
})  
export class AppModule {}
```

Lines 29-30 Explanation:

- `})` - Closes the @Module decorator configuration
- `export` - Makes this class importable by other files
- `class` - Keyword to define a class
- `AppModule` - The class name
- `{}` - Empty class body (all configuration is in the decorator)

- **Why we need it:** Defines the class that represents the module
 - **If deleted:** No root module exists
-

3. `app.controller.ts` - Root Controller

Location: `src/app.controller.ts`

Controllers handle incoming HTTP requests and return responses. This is the **root controller** for the base URL.

```
import { Controller, Get } from '@nestjs/common';
```

Line 1 Explanation:

- `Controller` - Decorator that marks a class as a controller
- `Get` - Decorator for HTTP GET method
- **Why we need it:** Essential decorators for creating REST endpoints
- **If deleted:** Cannot create controllers or GET endpoints

```
import { AppService } from '../app.service';
```

Line 2 Explanation:

- Imports the `AppService` for dependency injection
- **Why we need it:** Controller needs access to service methods
- **If deleted:** Cannot use `AppService` in this controller

```
@Controller()
```

Line 4 Explanation:

- `@Controller()` - Decorator marking this class as a controller
- Empty parentheses `()` - No route prefix (handles root / path)
- **Why we need it:** Tells NestJS this is a controller
- **If deleted:** Class is not recognized as a controller, routes won't work

```
export class AppController {
```

Line 5 Explanation:

- `export` - Makes class available to other files
- `class AppController` - Defines the controller class
- **Why we need it:** The actual controller class definition

- **If deleted:** No controller exists

```
constructor(private readonly appService: AppService) {}
```

Line 6 Explanation:

- **constructor** - Special method called when creating an instance
- **private** - Access modifier (only accessible within this class)
- **readonly** - Property cannot be reassigned after initialization
- **appService: AppService** - Parameter with type annotation, NestJS automatically injects an instance
- **{}** - Empty constructor body (injection is automatic)
- **Why we need it:** Dependency Injection - NestJS automatically provides AppService instance
- **If deleted:** Cannot access AppService, **this.appService** is undefined

```
@Get()
```

Line 8 Explanation:

- **@Get()** - Decorator that maps this method to HTTP GET requests
- Empty **()** - No additional path (responds to **/**)
- **Why we need it:** Defines what HTTP method triggers this function
- **If deleted:** Method doesn't respond to any HTTP requests

```
getHello(): string {
```

Line 9 Explanation:

- **getHello()** - Method name
- **: string** - Return type annotation (returns a string)
- **Why we need it:** The method that handles the request
- **If deleted:** No handler for GET **/** request

```
return this.appService.getHello();
```

Line 10 Explanation:

- **return** - Sends the result back as HTTP response
- **this.appService** - The injected service instance
- **.getHello()** - Calls the service method
- **Why we need it:** Delegates business logic to the service layer
- **If deleted:** Method returns nothing, endpoint returns empty response

```
}  
}
```

Lines 11-12 Explanation:

- Closes the method and class
 - **Why we need it:** Proper syntax
 - **If deleted:** Syntax error
-

4. `app.service.ts` - Root Service

Location: `src/app.service.ts`

Services contain **business logic** and are injected into controllers.

```
import { Injectable } from '@nestjs/common';
```

Line 1 Explanation:

- `Injectable` - Decorator that marks a class as injectable (can be used with dependency injection)
- **Why we need it:** Allows this service to be injected into other classes
- **If deleted:** Service cannot be injected, dependency injection fails

```
@Injectable()
```

Line 3 Explanation:

- `@Injectable()` - Marks this class as a provider that can be injected
- **Why we need it:** Required for NestJS dependency injection system
- **If deleted:** NestJS cannot inject this service into controllers

```
export class AppService {
```

Line 4 Explanation:

- Defines the service class
- **Why we need it:** The class that contains business logic
- **If deleted:** No service exists

```
  getHello(): string {  
    return 'Hello From TP02!';  
  }
```

Lines 5-7 Explanation:

- `getHello()` - Method name
- `: string` - Return type
- `return 'Hello From TP02!'` - Returns a simple string
- **Why we need it:** Contains the actual business logic
- **If deleted:** No logic to return, controller fails

```
}
```

Line 8 Explanation:

- Closes the class
- **Why we need it:** Proper syntax
- **If deleted:** Syntax error

5. `app.controller.spec.ts` - Unit Test File

Location: `src/app.controller.spec.ts`

This file contains **unit tests** for the AppController using Jest testing framework.

```
import { Test, TestingModule } from '@nestjs/testing';
```

Line 1 Explanation:

- `Test` - Utility class for creating testing modules
- `TestingModule` - Type representing a compiled test module
- `@nestjs/testing` - NestJS testing utilities package
- **Why we need it:** Provides tools to create isolated test environments
- **If deleted:** Cannot create test modules

```
import { AppController } from './app.controller';
```

Line 2 Explanation:

- Imports the controller we want to test
- **Why we need it:** Need access to the class we're testing
- **If deleted:** Cannot test the controller

```
import { AppService } from './app.service';
```

Line 3 Explanation:

- Imports the service (required as a dependency)
- **Why we need it:** Controller depends on this service
- **If deleted:** Cannot provide the dependency

```
describe('AppController', () => {
```

Line 5 Explanation:

- `describe` - Jest function to group related tests
- `'AppController'` - Test suite name
- `() => {` - Arrow function containing tests
- **Why we need it:** Organizes tests into logical groups
- **If deleted:** Tests are not grouped properly

```
let appController: AppController;
```

Line 6 Explanation:

- `let` - Variable declaration (can be reassigned)
- `appController: AppController` - Variable to hold controller instance
- **Why we need it:** Need to reference the controller in tests
- **If deleted:** Cannot access controller in tests

```
beforeEach(async () => {
```

Line 8 Explanation:

- `beforeEach` - Jest hook that runs before each test
- `async` - Makes the function asynchronous
- **Why we need it:** Sets up fresh test environment before each test
- **If deleted:** Setup code doesn't run, tests fail

```
const app: TestingModule = await Test.createTestingModule({  
  controllers: [AppController],  
  providers: [AppService],  
}).compile();
```

Lines 9-12 Explanation:

- `Test.createTestingModule()` - Creates a module for testing
- `controllers: [AppController]` - Controllers to include

- **providers:** [AppService] - Services to include
- **.compile()** - Compiles the module
- **Why we need it:** Creates isolated module with only what we need to test
- **If deleted:** No test module, cannot get controller instance

```
appController = app.get<AppController>(AppController);
```

Line 14 Explanation:

- **app.get<AppController>(AppController)** - Gets an instance of AppController from the test module
- **<AppController>** - Generic type parameter for type safety
- **Why we need it:** Retrieves the controller instance to test
- **If deleted:** **appController** is undefined

```
});
```

Line 15 Explanation:

- Closes the **beforeEach** function
- **Why we need it:** Proper syntax
- **If deleted:** Syntax error

```
describe('root', () => {
```

Line 17 Explanation:

- Nested describe block for root endpoint tests
- **Why we need it:** Further organizes tests by feature
- **If deleted:** Tests still work but less organized

```
it('should return "Hello World!"', () => {
```

Line 18 Explanation:

- **it** - Jest function defining a single test case
- **'should return "Hello World!"'** - Test description
- **Why we need it:** Defines what we're testing
- **If deleted:** No test exists

```
expect(appController.getHello()).toBe('Hello World!');
```

Line 19 Explanation:

- `expect()` - Jest assertion function
- `appController.getHello()` - Calls the method being tested
- `.toBe()` - Matcher checking exact equality
- `'Hello World!'` - Expected value
- **Note:** This test will FAIL because the actual return is `'Hello From TP02!'`
- **Why we need it:** Verifies the method returns expected value
- **If deleted:** No assertion, test passes without checking anything

```
    });  
  });  
});
```

Lines 20-22 Explanation:

- Closes all blocks
- **Why we need it:** Proper syntax
- **If deleted:** Syntax error

Database Layer

6. `receipts.entities.ts` - Database Entity

Location: `src/database/entities/receipts.entities.ts`

An **entity** represents a database table. TypeORM uses entities to create and manage database schemas.

```
import { Entity, PrimaryGeneratedColumn, Column } from 'typeorm';
```

Line 1 Explanation:

- `Entity` - Decorator that marks a class as a database table
- `PrimaryGeneratedColumn` - Decorator for auto-generated primary key columns
- `Column` - Decorator for regular database columns
- `typeorm` - The TypeORM library
- **Why we need it:** Required decorators for defining database structure
- **If deleted:** Cannot define entity, no database table

```
@Entity('receipts')
```

Line 3 Explanation:

- `@Entity('receipts')` - Marks this class as an entity

- **'receipts'** - The table name in the database
- **Why we need it:** Tells TypeORM this class maps to a database table
- **If deleted:** Class is not recognized as an entity

```
export class Receipt {
```

Line 4 Explanation:

- Defines the entity class
- **Why we need it:** The class that represents the table structure
- **If deleted:** No entity exists

```
@PrimaryGeneratedColumn('uuid')  
receiptId: string;
```

Lines 5-6 Explanation:

- **@PrimaryGeneratedColumn('uuid')** - Decorator for primary key with auto-generated UUID
- **'uuid'** - Specifies UUID format instead of auto-incrementing integer
- **receiptId: string** - Property name and type
- **UUID** - Universally Unique Identifier (e.g., **550e8400-e29b-41d4-a716-446655440000**)
- **Why we need it:** Every table needs a primary key to uniquely identify rows
- **If deleted:** Table has no primary key, cannot identify records

```
@Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })  
issuedAt: Date;
```

Lines 8-9 Explanation:

- **@Column({...})** - Defines a database column with options
- **type: 'timestamp'** - PostgreSQL timestamp type
- **default: () => 'CURRENT_TIMESTAMP'** - Default value is current time
- **() => 'CURRENT_TIMESTAMP'** - Arrow function returning SQL expression
- **issuedAt: Date** - When the receipt was issued
- **Why we need it:** Tracks when the receipt was created
- **If deleted:** No issuedAt column, lose important business data

```
@Column({ type: 'varchar', length: 255 })  
name: string;
```

Lines 11-12 Explanation:

- **type: 'varchar'** - Variable-length character string

- **length: 255** - Maximum 255 characters
- **name: string** - Receipt name/description
- **Why we need it:** Stores what the receipt is for
- **If deleted:** Cannot store receipt names

```
@Column({ type: 'decimal', precision: 10, scale: 2 })  
price: number;
```

Lines 14-15 Explanation:

- **type: 'decimal'** - Exact decimal number (good for money)
- **precision: 10** - Total digits allowed
- **scale: 2** - Digits after decimal point
- This allows values like **12345678.99**
- **Why we need it:** Stores the price with precision (avoiding floating-point errors)
- **If deleted:** Cannot store receipt prices

```
@Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })  
createdAt: Date;
```

Lines 17-18 Explanation:

- Auto-set timestamp when record is created
- **Why we need it:** Audit trail - when was the record created
- **If deleted:** Lose creation timestamp

```
@Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP',  
onUpdate: 'CURRENT_TIMESTAMP' })  
updatedAt: Date;
```

Lines 20-21 Explanation:

- **onUpdate: 'CURRENT_TIMESTAMP'** - Automatically updates when record is modified
- **Why we need it:** Tracks when record was last modified
- **If deleted:** Cannot track updates

```
}
```

Line 22 Explanation:

- Closes the class
- **Why we need it:** Proper syntax

Receipts Feature Module

7. `receipts.module.ts` - Feature Module

Location: `src/receipts/receipts.module.ts`

A **feature module** groups related functionality (controller, service, entities) together.

```
import { Module } from '@nestjs/common';
```

Line 1 Explanation:

- Imports the Module decorator
- **Why we need it:** Required to create a module
- **If deleted:** Cannot define the module

```
import { TypeOrmModule } from '@nestjs/typeorm';
```

Line 2 Explanation:

- Imports TypeORM module for database operations
- **Why we need it:** To use TypeORM repositories
- **If deleted:** Cannot access database in this module

```
import { Receipt } from '../database/entities/receipts.entities';
```

Line 3 Explanation:

- Imports the Receipt entity
- `../` - Goes up one directory level
- **Why we need it:** To register the entity with TypeORM
- **If deleted:** Cannot work with receipts table

```
import { ReceiptsController } from './receipts.controller';
```

Line 4 Explanation:

- Imports the receipts controller
- **Why we need it:** To register the controller
- **If deleted:** Receipts endpoints won't exist

```
import { ReceiptsService } from './receipts.service';
```

Line 5 Explanation:

- Imports the receipts service
- **Why we need it:** To register the service as a provider
- **If deleted:** Service won't be injectable

```
@Module({  
  imports: [TypeOrmModule.forFeature([Receipt])],
```

Lines 7-8 Explanation:

- `TypeOrmModule.forFeature([Receipt])` - Registers the Receipt entity for this module
- `forFeature` - Used in feature modules (vs `forRoot` in root module)
- `[Receipt]` - Array of entities this module uses
- **Why we need it:** Makes the Receipt repository available for injection
- **If deleted:** Cannot inject Receipt repository in service

```
  controllers: [ReceiptsController],
```

Line 9 Explanation:

- Registers the controller
- **Why we need it:** Makes routes available
- **If deleted:** No receipt endpoints

```
  providers: [ReceiptsService],
```

Line 10 Explanation:

- Registers the service as a provider
- **Why we need it:** Makes service injectable
- **If deleted:** Service cannot be injected

```
  exports: [ReceiptsService],
```

Line 11 Explanation:

- `exports` - Makes ReceiptsService available to other modules that import this module
- **Why we need it:** Allows other modules to use ReceiptsService
- **If deleted:** Other modules cannot use this service

```
  })  
  export class ReceiptsModule {}
```

Lines 12-13 Explanation:

- Defines the module class
 - **Why we need it:** The actual module class
 - **If deleted:** No module exists
-

8. `receipts.controller.ts` - API Endpoints

Location: `src/receipts/receipts.controller.ts`

This controller handles all **HTTP requests** for the `/receipts` endpoint.

```
import { Body, Controller, Delete, Get, Param, Patch, Post } from
 '@nestjs/common';
```

Line 1 Explanation:

- `Body` - Decorator to extract request body
- `Controller` - Marks class as controller
- `Delete` - HTTP DELETE method decorator
- `Get` - HTTP GET method decorator
- `Param` - Decorator to extract route parameters
- `Patch` - HTTP PATCH method decorator (partial update)
- `Post` - HTTP POST method decorator (create)
- **Why we need it:** All decorators needed for CRUD operations
- **If deleted:** Cannot create endpoints

```
import { ReceiptsService } from './receipts.service';
```

Line 2 Explanation:

- Imports the service for dependency injection
- **Why we need it:** Controller delegates to service
- **If deleted:** Cannot call service methods

```
import { CreateReceiptDto } from './dto/create-receipt.dto';
```

Line 3 Explanation:

- `DTO` - Data Transfer Object (defines shape of incoming data)
 - **Why we need it:** Type-safe data validation for creating receipts
 - **If deleted:** No type checking for create requests
-

```
import { UpdateReceiptDto } from '../dto/update-receipt.dto';
```

Line 4 Explanation:

- DTO for update operations
- **Why we need it:** Type-safe data validation for updates
- **If deleted:** No type checking for update requests

```
import { UseGuards } from '@nestjs/common';
```

Line 5 Explanation:

- **UseGuards** - Decorator to apply guards to routes
- **Why we need it:** To protect routes with authentication
- **If deleted:** Cannot apply API key guard

```
import { ApiKeyGuard } from 'src/common/guards/api-key.guard';
```

Line 6 Explanation:

- Imports the custom API key guard
- **Why we need it:** To use the guard
- **If deleted:** Cannot reference the guard

```
@UseGuards(ApiKeyGuard)
```

Line 8 Explanation:

- Applies the API key guard to ALL routes in this controller
- **Why we need it:** Protects all receipt endpoints with API key authentication
- **If deleted:** Endpoints are publicly accessible (security risk!)

```
@Controller('receipts')
```

Line 9 Explanation:

- **'receipts'** - Route prefix for all endpoints in this controller
- All routes become **/receipts**, **/receipts/:id**, etc.
- **Why we need it:** Groups all receipt routes under **/receipts**
- **If deleted:** Routes would be at root level

```
export class ReceiptsController {  
    constructor(private readonly receiptsService: ReceiptsService) {}  
}
```

Lines 10-11 Explanation:

- Injects the ReceiptsService
- `private readonly` - Service is private and cannot be reassigned
- **Why we need it:** Controller needs service for business logic
- **If deleted:** Cannot access service methods

```
@Get()  
findAll() {  
    return this.receiptsService.findAll();  
}
```

Lines 13-16 Explanation:

- `@Get()` - HTTP GET at `/receipts`
- `findAll()` - Method name
- `this.receiptsService.findAll()` - Delegates to service
- **Why we need it:** List all receipts endpoint
- **If deleted:** Cannot list receipts

```
@Get('/:id')  
findOne(@Param('id') id: string) {  
    return this.receiptsService.findOne(id);  
}
```

Lines 18-21 Explanation:

- `@Get('/:id')` - HTTP GET at `/receipts/:id` (e.g., `/receipts/abc-123`)
- `:id` - Route parameter placeholder
- `@Param('id')` - Extracts the `id` from the URL
- `id: string` - The extracted parameter as a string
- **Why we need it:** Get single receipt by ID
- **If deleted:** Cannot get specific receipt

```
@Post()  
create(@Body() dto: CreateReceiptDto) {  
    console.log('Creating receipt with data:', dto);  
    return this.receiptsService.create(dto);  
}
```

Lines 23-27 Explanation:

- `@Post()` - HTTP POST at `/receipts`
- `@Body()` - Extracts the request body
- `dto: CreateReceiptDto` - Body is validated against this DTO type
- `console.log(...)` - Debug logging
- **Why we need it:** Create new receipt endpoint
- **If deleted:** Cannot create receipts

```
@Patch('/:id')
update(@Param('id') id: string, @Body() dto: UpdateReceiptDto) {
  return this.receiptsService.update(id, dto);
}
```

Lines 29-32 Explanation:

- `@Patch('/:id')` - HTTP PATCH at `/receipts/:id`
- `PATCH` - HTTP method for partial updates
- Combines `@Param` and `@Body` to get both ID and update data
- **Why we need it:** Update existing receipt
- **If deleted:** Cannot update receipts

```
@Delete('/:id')
remove(@Param('id') id: string) {
  return this.receiptsService.remove(id);
}
}
```

Lines 34-38 Explanation:

- `@Delete('/:id')` - HTTP DELETE at `/receipts/:id`
- Removes a receipt by ID
- **Why we need it:** Delete receipt endpoint
- **If deleted:** Cannot delete receipts

9. `receipts.service.ts` - Business Logic

Location: `src/receipts/receipts.service.ts`

The service contains all **business logic** and **database operations**.

```
import { Injectable, NotFoundException } from '@nestjs/common';
```

Line 1 Explanation:

- `Injectable` - Allows this class to be injected

- **NotFoundException** - HTTP 404 exception class
- **Why we need it:** For DI and proper error handling
- **If deleted:** Service cannot be injected, cannot throw proper errors

```
import { InjectRepository } from '@nestjs/typeorm';
```

Line 2 Explanation:

- **InjectRepository** - Decorator to inject a TypeORM repository
- **Why we need it:** To get the Receipt repository for database operations
- **If deleted:** Cannot inject the repository

```
import { Repository } from 'typeorm';
```

Line 3 Explanation:

- **Repository** - TypeORM class that provides database methods
- **Why we need it:** Type annotation for the repository
- **If deleted:** No type for receiptRepo

```
import { Receipt } from '../database/entities/receipts.entities';
```

Line 4 Explanation:

- Imports the Receipt entity
- **Why we need it:** Type information for the repository
- **If deleted:** Cannot type the repository correctly

```
import { CreateReceiptDto } from '../dto/create-receipt.dto';  
import { UpdateReceiptDto } from '../dto/update-receipt.dto';
```

Lines 5-6 Explanation:

- Imports DTOs for method parameters
- **Why we need it:** Type safety for incoming data
- **If deleted:** Parameters lose type information

```
@Injectable()  
export class ReceiptsService {
```

Lines 8-9 Explanation:

- Marks class as injectable
- **Why we need it:** Allows NestJS DI system to manage this service
- **If deleted:** Cannot inject this service

```
constructor(  
  @InjectRepository(Receipt)  
  private readonly receiptRepo: Repository<Receipt>,  
) {}
```

Lines 10-13 Explanation:

- `constructor` - Constructor for dependency injection
- `@InjectRepository(Receipt)` - Tells NestJS to inject the Receipt repository
- `private readonly receiptRepo` - The repository instance
- `Repository<Receipt>` - Generic repository typed for Receipt entity
- **Why we need it:** Get access to database operations for Receipt table
- **If deleted:** Cannot perform database operations

```
async findAll() {  
  return this.receiptRepo.find({ order: { issuedAt: 'DESC' } });  
}
```

Lines 15-17 Explanation:

- `async` - Method returns a Promise
- `this.receiptRepo.find()` - Gets all records from receipts table
- `{ order: { issuedAt: 'DESC' } }` - Sorts by issuedAt descending (newest first)
- **Why we need it:** Retrieves all receipts from database
- **If deleted:** Cannot list receipts

```
async findOne(receiptId: string) {  
  const receipt = await this.receiptRepo.findOne({ where: { receiptId }  
});  
  if (!receipt) throw new NotFoundException('Receipt not found');  
  return receipt;  
}
```

Lines 19-23 Explanation:

- `receiptId: string` - The ID to search for
- `findOne({ where: { receiptId } })` - Finds single record matching the ID
- `{ receiptId }` - Shorthand for `{ receiptId: receiptId }`
- `if (!receipt)` - Checks if receipt was found
- `throw new NotFoundException(...)` - Throws HTTP 404 error if not found
- **Why we need it:** Gets a specific receipt or returns proper error

- **If deleted:** Cannot get single receipt

```
async create(dto: CreateReceiptDto) {  
  const receipt = this.receiptRepo.create({  
    issuedAt: new Date(),  
    name: dto.name,  
    price: dto.price,  
  });  
  return this.receiptRepo.save(receipt);  
}
```

Lines 25-32 Explanation:

- `this.receiptRepo.create({...})` - Creates a Receipt entity instance (not saved yet)
- `issuedAt: new Date()` - Sets current date/time
- `name: dto.name` - Takes name from DTO
- `price: dto.price` - Takes price from DTO
- `this.receiptRepo.save(receipt)` - Saves to database and returns the saved entity
- **Why we need it:** Creates new receipts in the database
- **If deleted:** Cannot create receipts

```
async update(receiptId: string, dto: UpdateReceiptDto) {  
  const receipt = await this.findOne(receiptId);  
  
  if (dto.issuedAt !== undefined) receipt.issuedAt = new  
Date(dto.issuedAt);  
  if (dto.name !== undefined) receipt.name = dto.name;  
  if (dto.price !== undefined) receipt.price = dto.price;  
  
  return this.receiptRepo.save(receipt);  
}
```

Lines 34-42 Explanation:

- `await this.findOne(receiptId)` - First finds the receipt (throws 404 if not found)
- `if (dto.X !== undefined)` - Only updates fields that were provided
- `!== undefined` - Allows setting values to `null` or empty string
- `new Date(dto.issuedAt)` - Converts string to Date object
- `this.receiptRepo.save(receipt)` - Saves the updated entity
- **Why we need it:** Updates existing receipts with partial data
- **If deleted:** Cannot update receipts

```
async remove(receiptId: string) {  
  const receipt = await this.findOne(receiptId);  
  await this.receiptRepo.remove(receipt);  
  return { deleted: true, receiptId };  
}
```

```
}  
}
```

Lines 44-49 Explanation:

- `await this.findOne(receiptId)` - Ensures receipt exists (throws 404 if not)
- `this.receiptRepo.remove(receipt)` - Deletes from database
- `return { deleted: true, receiptId }` - Returns confirmation object
- **Why we need it:** Deletes receipts from database
- **If deleted:** Cannot delete receipts

Data Transfer Objects (DTOs)

10. `create-receipt.dto.ts`

Location: `src/receipts/dto/create-receipt.dto.ts`

DTOs define the **shape of incoming data** and provide **validation**.

```
import { IsDateString, IsNotEmpty, IsNumber, IsString, Min } from 'class-validator';
```

Line 1 Explanation:

- `IsDateString` - Validates ISO 8601 date strings (currently commented out)
- `IsNotEmpty` - Validates field is not empty
- `IsNumber` - Validates field is a number
- `IsString` - Validates field is a string
- `Min` - Validates minimum value
- `class-validator` - Validation library
- **Why we need it:** Decorators for validating request data
- **If deleted:** No validation occurs

```
export class CreateReceiptDto {
```

Line 3 Explanation:

- Defines the DTO class
- **Why we need it:** Shape of data for creating receipts
- **If deleted:** No type for create requests

```
// @IsDateString()  
// issuedAt: string;
```

Lines 4-5 Explanation:

- Commented out - issuedAt is auto-set in the service
- **Why commented:** User doesn't need to provide this
- **If uncommented:** User would need to provide issuedAt

```
@IsString()  
@NotEmpty()  
name: string;
```

Lines 7-9 Explanation:

- `@IsString()` - Validates name is a string
- `@NotEmpty()` - Validates name is not empty
- `name: string` - The property
- **Why we need it:** Ensures name is provided and valid
- **If deleted:** Name could be missing or wrong type

```
@IsNumber()  
@Min(0)  
price: number;  
}
```

Lines 11-14 Explanation:

- `@IsNumber()` - Validates price is a number
- `@Min(0)` - Price must be 0 or greater
- `price: number` - The property
- **Why we need it:** Ensures valid price
- **If deleted:** Price could be negative or wrong type

11. update-receipt.dto.ts

Location: `src/receipts/dto/update-receipt.dto.ts`

DTO for **partial updates** to receipts.

```
import { IsDateString, IsNumber, IsOptional, IsString, Min } from 'class-validator';
```

Line 1 Explanation:

- `IsOptional` - Marks field as optional (can be undefined)
- Other imports same as create DTO
- **Why we need it:** All validation decorators needed

- **If deleted:** Cannot validate

```
export class UpdateReceiptDto {
```

Line 3 Explanation:

- Defines the update DTO
- **Why we need it:** Shape of data for updates
- **If deleted:** No type for update requests

```
@IsOptional()  
@IsDateString()  
issuedAt?: string;
```

Lines 4-6 Explanation:

- `@IsOptional()` - Field doesn't have to be provided
- `@IsDateString()` - If provided, must be valid date string
- `issuedAt?:` - The `?` makes TypeScript property optional
- **Why we need it:** Allows updating issuedAt optionally
- **If deleted:** Cannot update issuedAt

```
@IsOptional()  
@IsString()  
name?: string;
```

Lines 8-10 Explanation:

- Optional name field for updates
- **Why we need it:** Allows updating name optionally
- **If deleted:** Cannot update name

```
@IsOptional()  
@IsNumber()  
@Min(0)  
price?: number;  
}
```

Lines 12-16 Explanation:

- Optional price field for updates
- **Why we need it:** Allows updating price optionally
- **If deleted:** Cannot update price

Common Utilities

12. `api-key.guard.ts` - Security Guard

Location: `src/common/guards/api-key.guard.ts`

A **guard** determines whether a request should be handled. This guard checks for a valid API key.

```
import { CanActivate, ExecutionContext, Injectable, UnauthorizedException }  
from '@nestjs/common';
```

Line 1 Explanation:

- `CanActivate` - Interface that guards must implement
- `ExecutionContext` - Object containing request/response information
- `Injectable` - Allows guard to be injected
- `UnauthorizedException` - HTTP 401 error class
- **Why we need it:** Required for creating guards
- **If deleted:** Cannot create the guard

```
@Injectable()  
export class ApiKeyGuard implements CanActivate {
```

Lines 3-4 Explanation:

- `@Injectable()` - Makes guard injectable
- `implements CanActivate` - Must implement the `CanActivate` interface
- **Why we need it:** Guard must follow the `CanActivate` contract
- **If deleted:** Guard won't work

```
  canActivate(context: ExecutionContext): boolean {
```

Line 5 Explanation:

- `canActivate` - Required method from `CanActivate` interface
- `context: ExecutionContext` - Contains request information
- `: boolean` - Returns true (allow) or false (deny)
- **Why we need it:** This method decides if request proceeds
- **If deleted:** Guard has no logic

```
    const req = context.switchToHttp().getRequest<Request & { headers: any  
}>();
```

Line 6 Explanation:

- `context.switchToHttp()` - Switches to HTTP context (vs WebSocket, etc.)
- `.getRequest<...>()` - Gets the HTTP request object
- `Request & { headers: any }` - Type with headers property
- **Why we need it:** Access to request headers
- **If deleted:** Cannot read headers

```
const apiKey = req.headers['x-api-key'];
```

Line 7 Explanation:

- Extracts the `x-api-key` header from the request
- `['x-api-key']` - Bracket notation for hyphenated property
- **Why we need it:** Gets the API key from request
- **If deleted:** Cannot check API key

```
if (!apiKey || apiKey !== process.env.API_KEY) {  
    throw new UnauthorizedException('Invalid API key');  
}
```

Lines 9-11 Explanation:

- `!apiKey` - Checks if API key is missing
- `apiKey !== process.env.API_KEY` - Checks if key doesn't match
- `throw new UnauthorizedException(...)` - Returns HTTP 401 error
- **Why we need it:** Rejects invalid requests
- **If deleted:** All requests pass through (security hole!)

```
    return true;  
  }  
}
```

Lines 12-14 Explanation:

- `return true` - Allows the request to proceed
- **Why we need it:** Signals the request is valid
- **If deleted:** Valid requests also fail

13. `logging.interceptor.ts` - Request Logging

Location: `src/common/interceptors/logging.interceptor.ts`

An **interceptor** wraps around route handlers to add extra logic before/after.

```
import {
  CallHandler,
  ExecutionContext,
  Injectable,
  NestInterceptor,
} from '@nestjs/common';
```

Lines 1-6 Explanation:

- **CallHandler** - Interface to call the next handler in the chain
- **ExecutionContext** - Request context information
- **Injectable** - For dependency injection
- **NestInterceptor** - Interface that interceptors must implement
- **Why we need it:** Required for creating interceptors
- **If deleted:** Cannot create interceptor

```
import { Observable } from 'rxjs';
```

Line 7 Explanation:

- **Observable** - RxJS type for asynchronous data streams
- **Why we need it:** Interceptors work with Observables
- **If deleted:** Cannot return proper type

```
import { tap } from 'rxjs/operators';
```

Line 8 Explanation:

- **tap** - RxJS operator that performs side effects
- **Why we need it:** Execute logging without modifying the response
- **If deleted:** Cannot add logging logic

```
@Injectable()
export class LoggingInterceptor implements NestInterceptor {
```

Lines 10-11 Explanation:

- Defines the interceptor class
- **implements NestInterceptor** - Must follow this interface
- **Why we need it:** Interceptor definition
- **If deleted:** No interceptor

```
    intercept(context: ExecutionContext, next: CallHandler): Observable<any>
    {
```

Line 12 Explanation:

- `intercept` - Required method from `NestInterceptor`
- `context` - Request context
- `next: CallHandler` - The next handler (controller method)
- `Observable<any>` - Returns an observable stream
- **Why we need it:** This is where interception logic lives
- **If deleted:** Interceptor has no functionality

```
    const req = context.switchToHttp().getRequest();
    const { method, url } = req;
```

Lines 13-14 Explanation:

- Gets the HTTP request object
- `{ method, url }` - Destructures method (GET, POST, etc.) and URL
- **Why we need it:** Need request details for logging
- **If deleted:** Cannot log request info

```
    const start = Date.now();
```

Line 16 Explanation:

- `Date.now()` - Current timestamp in milliseconds
- **Why we need it:** To calculate response time
- **If deleted:** Cannot measure duration

```
    return next.handle().pipe(
      tap(() => {
        const ms = Date.now() - start;
        console.log(`[HTTP] ${method} ${url} - ${ms}ms`);
      }),
    );
```

Lines 17-22 Explanation:

- `next.handle()` - Calls the actual route handler
- `.pipe(...)` - Chains RxJS operators
- `tap(...)` - Runs side effect when response is ready
- `Date.now() - start` - Calculates elapsed time

- `console.log(...)` - Logs the request info and duration
- **Template literal** - String with `${...}` interpolation
- **Why we need it:** Logs every HTTP request with timing
- **If deleted:** No request logging

```
  }  
}
```

Lines 23-24 Explanation:

- Closes method and class
- **Why we need it:** Proper syntax

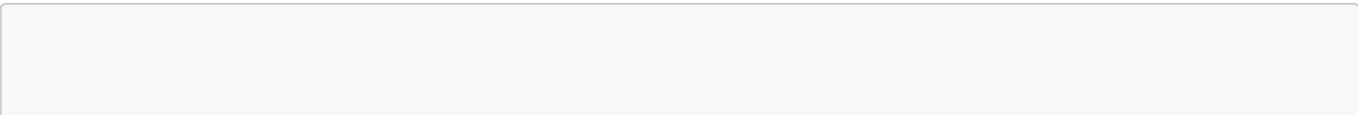
Code Relationships and Flow

Request Flow Diagram





Module Dependency Graph



/

Source File	Connects To	Relationship
receipts.module.ts	receipts.controller.ts	Registers controller
receipts.module.ts	receipts.service.ts	Registers provider
receipts.module.ts	receipts.entities.ts	Registers entity for repository
receipts.controller.ts	receipts.service.ts	Injects service (DI)
receipts.controller.ts	create-receipt.dto.ts	Uses for validation
receipts.controller.ts	update-receipt.dto.ts	Uses for validation
receipts.controller.ts	api-key.guard.ts	Applies guard
receipts.service.ts	receipts.entities.ts	Injects repository (DI)
receipts.service.ts	create-receipt.dto.ts	Uses as parameter type
receipts.service.ts	update-receipt.dto.ts	Uses as parameter type

Keyword Glossary

Keyword	Meaning
import	Brings external code into the file
export	Makes code available to other files
class	Blueprint for creating objects
constructor	Special method called when creating class instance
async	Marks function as asynchronous (returns Promise)
await	Waits for Promise to resolve
const	Declares constant (cannot be reassigned)
let	Declares variable (can be reassigned)
return	Sends value back from function
new	Creates new instance of a class
this	Reference to current object instance
private	Access modifier - only accessible within class
readonly	Property cannot be changed after initialization
@Decorator()	Function that modifies class/method/property behavior
@Module()	Marks class as NestJS module
@Controller()	Marks class as HTTP request handler
@Injectable()	Marks class as injectable provider

Keyword	Meaning
<code>@Get()</code>	Maps method to HTTP GET requests
<code>@Post()</code>	Maps method to HTTP POST requests
<code>@Patch()</code>	Maps method to HTTP PATCH requests
<code>@Delete()</code>	Maps method to HTTP DELETE requests
<code>@Body()</code>	Extracts request body
<code>@Param()</code>	Extracts route parameters
<code>@UseGuards()</code>	Applies guard to routes
<code>@Entity()</code>	Marks class as database entity
<code>@Column()</code>	Marks property as database column
<code>@PrimaryGeneratedColumn()</code>	Auto-generated primary key
<code>@InjectRepository()</code>	Injects TypeORM repository
<code>implements</code>	Class fulfills interface contract
<code>extends</code>	Class inherits from another
<code>interface</code>	Defines object shape (contract)
<code>type</code>	Defines type alias
<code>?:</code>	Optional property in TypeScript
<code>??</code>	Nullish coalescing (use right if left is null/undefined)
<code> </code>	Logical OR (use right if left is falsy)
<code>=></code>	Arrow function syntax
<code>Promise</code>	Object representing eventual completion
<code>Observable</code>	RxJS stream of values over time

Summary

This NestJS application follows a **modular architecture**:

1. **main.ts** - Entry point that bootstraps the app with global middleware
2. **app.module.ts** - Root module that configures database and imports features
3. **Feature Modules** (ReceiptsModule) - Encapsulate related functionality
4. **Controllers** - Handle HTTP requests and delegate to services
5. **Services** - Contain business logic and database operations
6. **Entities** - Define database schema using TypeORM
7. **DTOs** - Validate and transform incoming request data
8. **Guards** - Protect routes with authentication
9. **Interceptors** - Add cross-cutting concerns like logging

The application uses **Dependency Injection** throughout, where NestJS automatically provides required dependencies to classes through their constructors. This makes the code testable, maintainable, and loosely coupled.